

Getting Started with RDFLib.jl

Simon Frost

Introduction

RDFLib.jl is a comprehensive Julia package for working with [RDF](#) (Resource Description Framework) data — the foundational data model for the Semantic Web and Linked Data. It provides tools for creating, querying, serializing, and reasoning over RDF graphs.

This vignette walks through the basics: installing the package, creating your first graph, and performing simple operations.

Installation

```
using Pkg
Pkg.add(url="https://github.com/JuliaKnowledge/RDFLib.jl")
```

Loading the Package

```
using RDFLib
```

Your First RDF Graph

An RDF graph is a set of *triples* — statements of the form **subject** → **predicate** → **object**. Let's model some facts about a person.

```

# Create an empty graph
g = RDFGraph()

# Define a namespace for our vocabulary
ex = Namespace("http://example.org/")

# Add triples about Alice
add!(g, Triple(ex("alice"), RDF.type, FOAF("Person")))
add!(g, Triple(ex("alice"), FOAF("name"), Literal("Alice")))
add!(g, Triple(ex("alice"), FOAF("age"), Literal(30)))
add!(g, Triple(ex("alice"), FOAF("knows"), ex("bob")))

# Add triples about Bob
add!(g, Triple(ex("bob"), RDF.type, FOAF("Person")))
add!(g, Triple(ex("bob"), FOAF("name"), Literal("Bob")))

println("Graph has $(length(g)) triples")

```

Graph has 6 triples

Querying the Graph

You can query the graph by matching triple patterns. Use `nothing` as a wildcard.

```

# Find all people
println("People in the graph:")
for t in triples(g, (nothing, RDF.type, FOAF("Person")))
    println("  ", t.subject)
end

```

People in the graph:
 URIRef("http://example.org/alice")
 URIRef("http://example.org/bob")

```

# Find everything about Alice
println("\nFacts about Alice:")
for t in triples(g, (ex("alice"), nothing, nothing))
    println("  ", t.predicate, " → ", t.object)
end

```

Facts about Alice:

```
URIRef("http://www.w3.org/1999/02/22-rdf-syntax-ns#type") → URIRef("http://xmlns.com/foaf/")
URIRef("http://xmlns.com/foaf/0.1/name") → Literal("Alice")
URIRef("http://xmlns.com/foaf/0.1/age") → Literal("30", datatype=URIRef("http://www.w3.org/"))
URIRef("http://xmlns.com/foaf/0.1/knows") → URIRef("http://example.org/bob")
```

```
# Use convenience functions
println("\nAlice knows:")
for obj in objects(g, ex("alice"), FOAF("knows"))
    println("  ", obj)
end
```

Alice knows:

```
URIRef("http://example.org/bob")
```

Serialization

RDFLib.jl supports many RDF serialization formats. Let's output our graph in Turtle format:

```
ttl = serialize(g, TurtleFormat())
println(ttl)
```

```
@prefix ns1: <http://example.org/> .
@prefix ns2: <http://xmlns.com/foaf/0.1/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix skos: <http://www.w3.org/2004/02/skos/core#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
ns1:alice a ns2:Person ;
    ns2:age 30 ;
    ns2:knows ns1:bob ;
    ns2:name "Alice" .
```

```
ns1:bob a ns2:Person ;
    ns2:name "Bob" .
```

And in N-Triples format (one triple per line, fully explicit):

```
nt = serialize(g, NTriplesFormat())
println(nt)
```

```
<http://example.org/alice> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/foaf/0.1/Person> .
<http://example.org/alice> <http://xmlns.com/foaf/0.1/name> "Alice" .
<http://example.org/alice> <http://xmlns.com/foaf/0.1/age> "30"^^<http://www.w3.org/2001/XMLSchema#integer> .
<http://example.org/alice> <http://xmlns.com/foaf/0.1/knows> <http://example.org/bob> .
<http://example.org/bob> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/foaf/0.1/Person> .
<http://example.org/bob> <http://xmlns.com/foaf/0.1/name> "Bob" .
```

Parsing RDF Data

You can parse RDF data from strings:

```
turtle_data = """
@prefix ex: <http://example.org/> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .

ex:carol a foaf:Person ;
    foaf:name "Carol" ;
    foaf:knows ex:alice .
"""

g2 = parse_rdf(turtle_data, TurtleFormat())
println("Parsed $(length(g2)) triples about Carol")
```

Parsed 3 triples about Carol

Merging Graphs

Combine graphs easily:

```
merged = merge_graphs(g, g2)
println("Merged graph has $(length(merged)) triples")

# Find all people in the merged graph
println("\nAll people:")
for t in triples(merged, (nothing, RDF.type, FOAF("Person")))
```

```

    name_triples = collect(triples(merged, (t.subject, FOAF("name"), nothing)))
    name = isempty(name_triples) ? "unknown" : name_triples[1].object
    println("  $(t.subject) - $name")
end

```

Merged graph has 9 triples

All people:

```

URIRef("http://example.org/alice") - Literal("Alice")
URIRef("http://example.org/carol") - Literal("Carol")
URIRef("http://example.org/bob") - Literal("Bob")

```

SPARQL Queries

RDFLib.jl includes a built-in SPARQL query engine:

```

results = sparql_query(merged, """
    PREFIX foaf: <http://xmlns.com/foaf/0.1/>
    SELECT ?person ?name WHERE {
        ?person a foaf:Person .
        ?person foaf:name ?name .
    }
    ORDER BY ?name
""")

for row in results
    println("$(row["name"]) - $(row["person"])")
end

```

```

Literal("Alice") - URIRef("http://example.org/alice")
Literal("Bob") - URIRef("http://example.org/bob")
Literal("Carol") - URIRef("http://example.org/carol")

```

What's Next?

This was a quick taste of RDFLib.jl. Subsequent vignettes cover:

- **RDF Terms** — URIs, literals, blank nodes, and data types in depth
- **Building Graphs** — advanced graph construction patterns
- **Namespaces** — managing and creating vocabularies

- **Serialization** — all supported formats (Turtle, JSON-LD, RDF/XML, N3, etc.)
- **SPARQL** — full query and update support
- **Collections & Containers** — RDF lists and containers
- **Datasets** — named graphs and quads
- **Inference** — RDFS and OWL entailment
- **N3 Reasoning** — forward/backward chaining with the built-in reasoner
- **SHACL Validation** — validating graphs against shape constraints
- **Store Backends** — SQLite, DuckDB, LMDB, and remote SPARQL endpoints
- **Tabular Mapping** — convert DataFrames/CSV to RDF with OTTR templates
- **Datalog** — semi-naive bottom-up reasoning
- **ProbLog** — probabilistic logic programming with exact BDD inference
- **Property Paths** — navigate graphs with SPARQL path expressions
- **Graph Utilities** — isomorphism, CBD, VoID, visualization
- **Ecology Example** — an advanced end-to-end case study modeling a marine food web
- **Ash Dieback** — modeling UK ash dieback disease spread and ecological impact
- **Bayesian Belief Networks** — BBN analysis of species interaction networks using bbnet data
- **Epidemiology** — comprehensive infectious disease surveillance and outbreak analysis